



ÉCOLE NATIONALE DE LA STATISTIQUE
ET DE L'ANALYSE DE L'INFORMATION

PROJET INFORMATIQUE

Groupe 18

RAPPORT D'ANALYSE ET DE CONCEPTION

Concurrent SNCF

Une plateforme de vente de trajets ferroviaires concurrente de
la SNCF

Étudiants :

YVANO Maxime

VAN HECKE Jonas

SIDOBRE Noé

OUATTARA Youssouf Adam

ARRAJI Salma

Tuteur :

Kévin LEROY

Année 2026-2027

Préambule

Dans ce rapport, nous présentons **Concurrent SNCF**, notre application de vente de trajets ferroviaires réalisée dans le cadre de notre projet informatique en deuxième année. Celle-ci s'appuie sur les données fournies par l'API SNCF (Navitia) et a pour objectif de mettre en pratique les principes de programmation et de modélisation acquis au cours de notre scolarité.

Contexte. Nous sommes développeurs pour une nouvelle société de transport ferroviaire qui exploite le réseau SNCF. Notre mission est de mettre en place une solution permettant au service commercial de notre entreprise de proposer des trajets en train à ses clients.

Objectif. Développer une API permettant, selon le niveau d'habilitation de l'utilisateur (CLIENT, COLLABORATEUR ou ADMIN) :

- F1 — Gestion des comptes utilisateur : Cette fonctionnalité permet à tout utilisateur souhaitant accéder à l'application soit créer son compte à l'aide d'un mot de passe et de son nom, soit se connecter s'il avait déjà effectué cette étape dans le passé ; une fois le compte créé, l'utilisateur a la possibilité d'être soit un collaborateur, soit un administrateur, soit un client et dans le cas où celui-ci est un administrateur, il peut gérer le niveau d'habilitation des comptes sauf celui des autres comptes administrateurs.
- F2 — Gestion des lignes d'exploitation : Des lignes d'exploitation entre deux gares (une gare de départ et une gare d'arrivée) sont créées par les collaborateurs qui récupéreront ces informations sur l'API de la SNCF, cette dernière qui nous aidera lorsqu'une saisie sera faite au niveau du moteur de recherche de s'assurer que la gare saisie existe réellement. Le collaborateur a également la possibilité de supprimer une ligne d'exploitation.
- F3 — Gestion des trajets proposés : Cette fonctionnalité permet au collaborateur de planifier des trajets sur une ligne d'exploitation, celle-ci mentionnera la date, l'heure, le nombre de places disponibles et le tarif. Elle nous permet de nous assurer qu'il est possible d'effectuer un trajet entre deux gares tout en calculant le temps de trajet en train via l'API de la SNCF. Le collaborateur peut également modifier un trajet planifié et le supprimer.
- F4 — Recherche d'un trajet : Cette fonctionnalité permettra aux clients de pouvoir rechercher les trajets possibles à partir d'une gare de départ et d'une gare d'arrivée tout en lui fournissant une aide à la saisie des gares pour faciliter la recherche des noms des gares pour les clients. Elle permet au client de pouvoir visualiser la gare de départ, d'arrivée, la date et l'heure d'arrivée et une estimation du temps de trajet.
- F5 — Positionnement commercial (fonctionnalité différenciante) : Cette fonctionnalité permet de se démarquer par rapport aux autres sites de réservation de billets de train. Cette fonctionnalité est le fait de renvoyer une liste de tous les trajets optimaux lors d'une recherche, que ce soit en termes de durée de trajet, de prix, de nombre de correspondances ou d'heure d'arrivée.
- FO1 — Réservation de trajet : Cette fonctionnalité optionnelle permet à l'utilisateur de réserver un trajet en plus de le rechercher. L'utilisateur pourra aussi savoir

le nombre de places restantes dans le train. De plus, il aura la capacité d'annuler son trajet

- FO2 — Classes de voyage : Cette fonctionnalité permettra à l'utilisateur de choisir entre la classe standard et la classe premium pour son trajet.

Le détail de sa conception figure dans la suite de ce document.

Sommaire

1	Organisation	4
1.1	Cahier des charges	4
1.2	Étapes de réalisation	5
1.2.1	Diagramme de Gantt	5
2	Modélisation UML	6
2.1	Diagramme de cas d'utilisation	6
2.2	Diagramme de classes	7
2.3	Diagramme structure projet	9
2.4	Diagramme de séquence	11
3	Endpoints de l'API	13
3.1	Endpoints consommés de l'API SNCF	13
3.2	Endpoints exposés par notre API	13

Chapitre 1

Organisation

1.1 Cahier des charges

Présentation d'ensemble du projet Concurrent SNCF

Dans le cadre de notre projet d'informatique de deuxième année, il nous a été demandé de concevoir une solution logicielle destinée à une société de transport ferroviaire nouvellement créée dénommée **Concurrent SNCF**, exploitant le réseau SNCF. Celle-ci s'adresse à plusieurs utilisateurs dont les **clients** souhaitant rechercher et réserver des trajets en train, les **collaborateurs** de l'entreprise chargés de définir l'offre commerciale : quelles liaisons sont exploitées, à quels horaires, à quel tarif et avec quelle capacité.

Le développement de l'interface utilisateur étant pris en charge par les webmasters de l'entreprise, notre périmètre porte exclusivement sur la réalisation d'une **API**. Celle-ci expose l'ensemble des fonctionnalités de l'application sous forme de points d'entrée REST, documentés et interrogeables indépendamment de toute interface graphique.

Deux sources de données complémentaires seront notamment utilisées pour constituer l'application. D'une part, l'**API SNCF** (Navitia), qui fournit le référentiel des gares existantes ainsi que le calcul des temps de trajet réels entre deux gares ; ces données ne sont ni créées ni modifiables par notre application. D'autre part, une **base de données propre à l'application**, développée pour les besoins du projet, qui stocke les données métier que nous maîtrisons : les comptes utilisateurs et leurs niveaux d'habilitation, les lignes d'exploitation définies par l'entreprise, les trajets planifiés sur ces lignes, ainsi que les réservations effectuées par les clients.

Utilisateurs de l'application

Trois différents niveaux d'habilitation sont distingués dans l'accès aux fonctionnalités de l'API de Concurrent SNCF : les clients, les collaborateurs et les administrateurs. Chaque utilisateur dispose d'un compte applicatif créé à partir d'un nom d'utilisateur et d'un mot de passe. Les clients peuvent notamment rechercher les trajets proposés et, dans le cadre de la fonctionnalité de réservation, réserver, consulter et annuler leurs trajets. Les collaborateurs sont chargés de la gestion de l'offre ferroviaire : ils peuvent créer, modifier et supprimer les lignes d'exploitation ainsi que planifier les trajets associés. Enfin, les administrateurs assurent la gestion des habilitations des utilisateurs en pouvant augmenter ou diminuer leur niveau d'accès. Ils ne peuvent toutefois pas modifier le niveau d'habilitation des autres administrateurs. Cette organisation permet ainsi de contrôler l'accès

aux différentes fonctionnalités de l'application selon le rôle de chaque utilisateur et de sécuriser la gestion des données.

Outils de développement

L'application sera développée en Python. La gestion et le stockage des données seront assurés par PostgreSQL. Enfin, afin de faciliter la coordination et le travail collaboratif entre les membres de l'équipe, nous utiliserons Github pour centraliser le code et suivre les différentes modifications apportées au projet.

1.2 Étapes de réalisation

La réalisation du projet a été organisée en plusieurs étapes, de l'analyse du besoin à la conception et au développement de l'application.

1.2.1 Diagramme de Gantt

Afin d'organiser les différentes phases du projet, nous avons établi un planning prévisionnel permettant de répartir les principales étapes d'analyse, de conception et de développement sur la durée du projet.

La première partie du projet est dédiée à l'analyse de notre sujet et la réalisation de diagrammes afin d'orienter le développement de notre application. La deuxième se concentre sur le développement de l'application et l'écriture du rapport final. Puis la dernière partie sera dédiée à la préparation pour la soutenance.

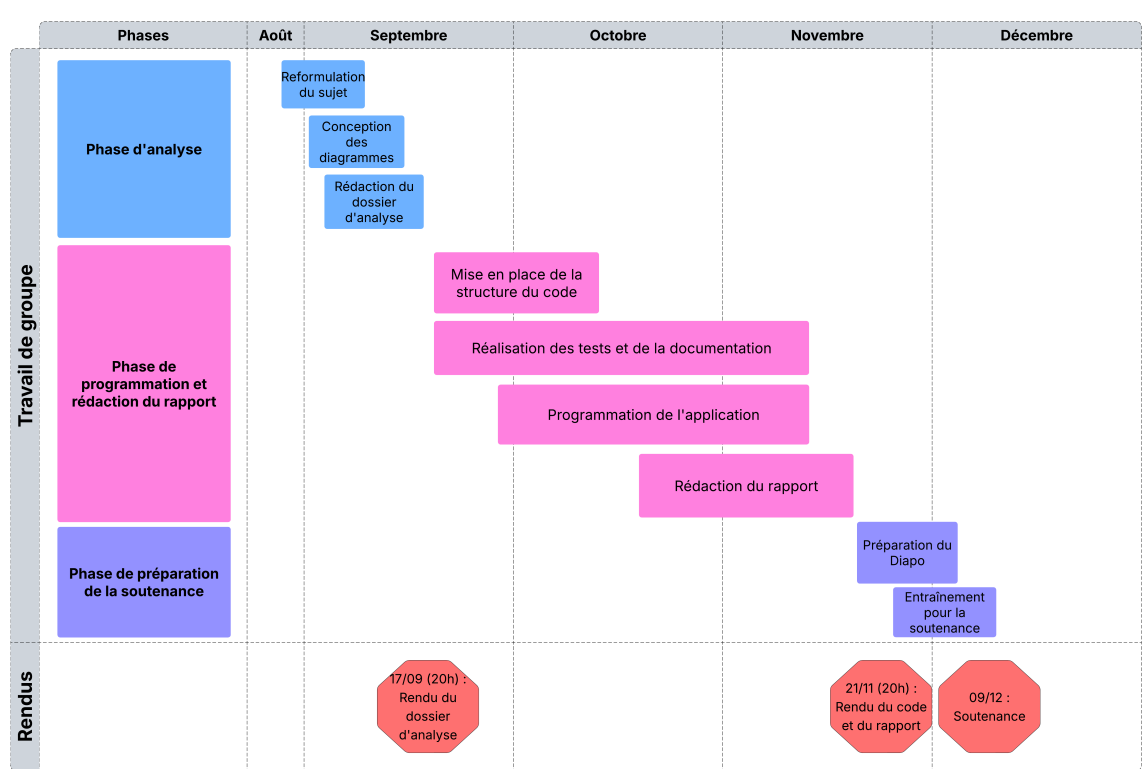


FIGURE 1.1 – Diagramme de Gantt : Phase de conception et de développement

Chapitre 2

Modélisation UML

2.1 Diagramme de cas d'utilisation

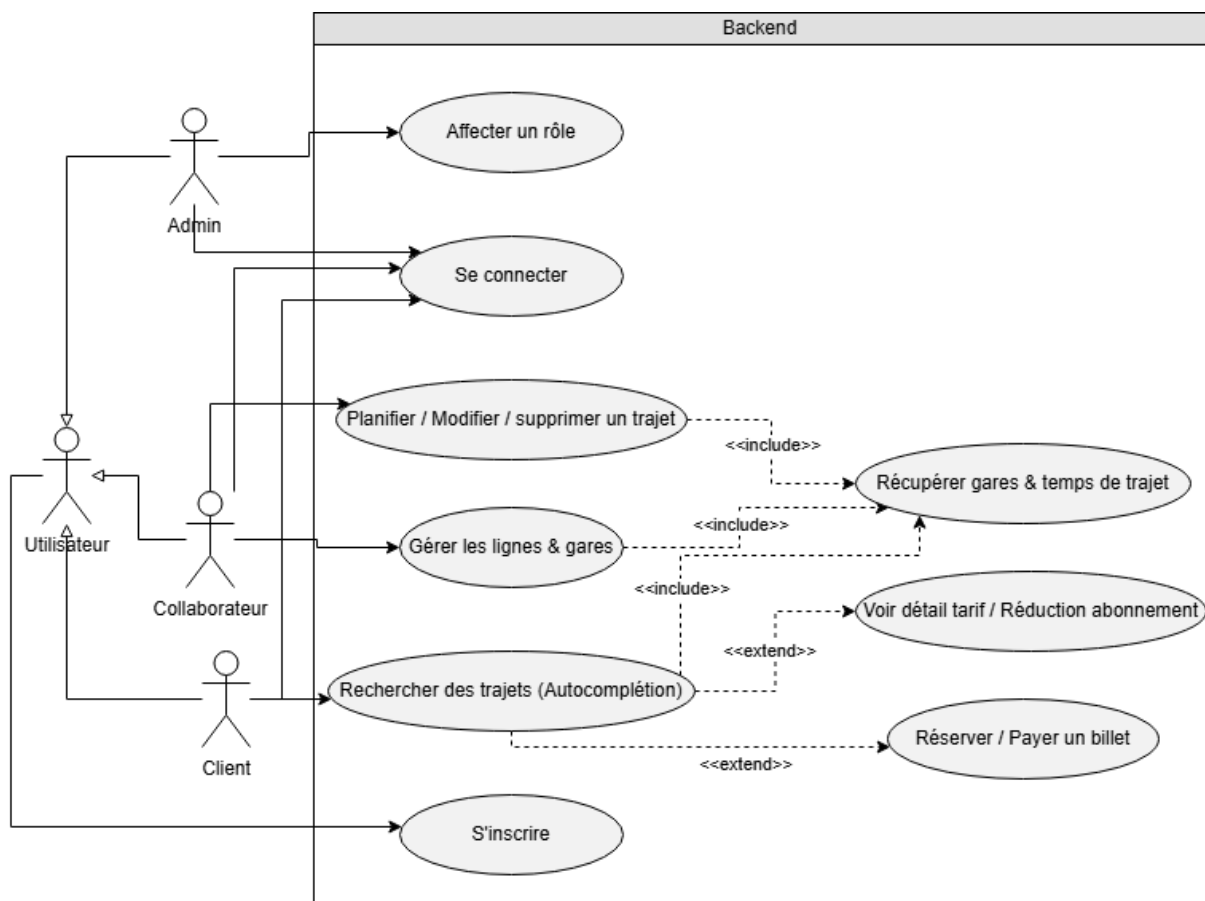


FIGURE 2.1 – Diagramme de cas d'utilisation

Ce diagramme de cas d'utilisation délimite la zone fonctionnelle du backend et structure les interactions selon le profil de l'utilisateur et avec l'API externe de la *SNCF*.

Les utilisateurs de l'application ont trois rôles, d'où le mécanisme d'héritage mis en place. Un **Utilisateur** crée son compte, puis il se connecte avec son rôle, parmi les **Admins**, les **Collaborateurs** et les **Clients**, et cela lui donne accès à ses possibilités dédiées.

- Le **Client** dispose des fonctionnalités de recherche, de consultation des tarifs et de réservation.
- Le **Collaborateur** s'occupe du réseau et de la planification des trajets.
- L'**Admin** supervise la gestion globale des rôles.

Pour rechercher un trajet, agir sur les trajets et gérer les gares et lignes, il faut récupérer toutes les informations des gares, des horaires des trajets, etc. à partir de **l'API SNCF**.

2.2 Diagramme de classes

Ce diagramme détaille les classes de l'application, leurs méthodes et les entités manipulées. Il s'organise en quatre ensembles, dont l'articulation est présentée dans la section suivante.

Chaque **controller** expose les points d'entrée *REST* d'une fonctionnalité et se contente de transmettre les requêtes : le **LineController** propose ainsi les opérations de consultation, de création, de modification et de suppression d'une ligne, correspondant directement aux exigences de la fonctionnalité **F2**.

Les **services** portent la logique métier et reprennent les mêmes opérations, enrichies des traitements propres au domaine. Le **TripService** sollicite le **SNCFClientService** pour vérifier l'existence d'une liaison ferroviaire et en calculer la durée lors de la planification d'un trajet (**F3**).

Les **repositories** exposent les opérations élémentaires de persistance sur une entité unique. Le **TripRepository** comporte en outre une méthode **search** dédiée à la recherche de trajets par gares et par date (**F4**).

Le **modèle métier** regroupe enfin les cinq entités persistées en base : **User**, **Line**, **Station**, **Trip** et **Booking**.

Le diagramme détaille donc les classes de l'application, leurs méthodes et les entités manipulées. Il s'organise en quatre ensembles, dont l'articulation est présentée dans la section suivante.

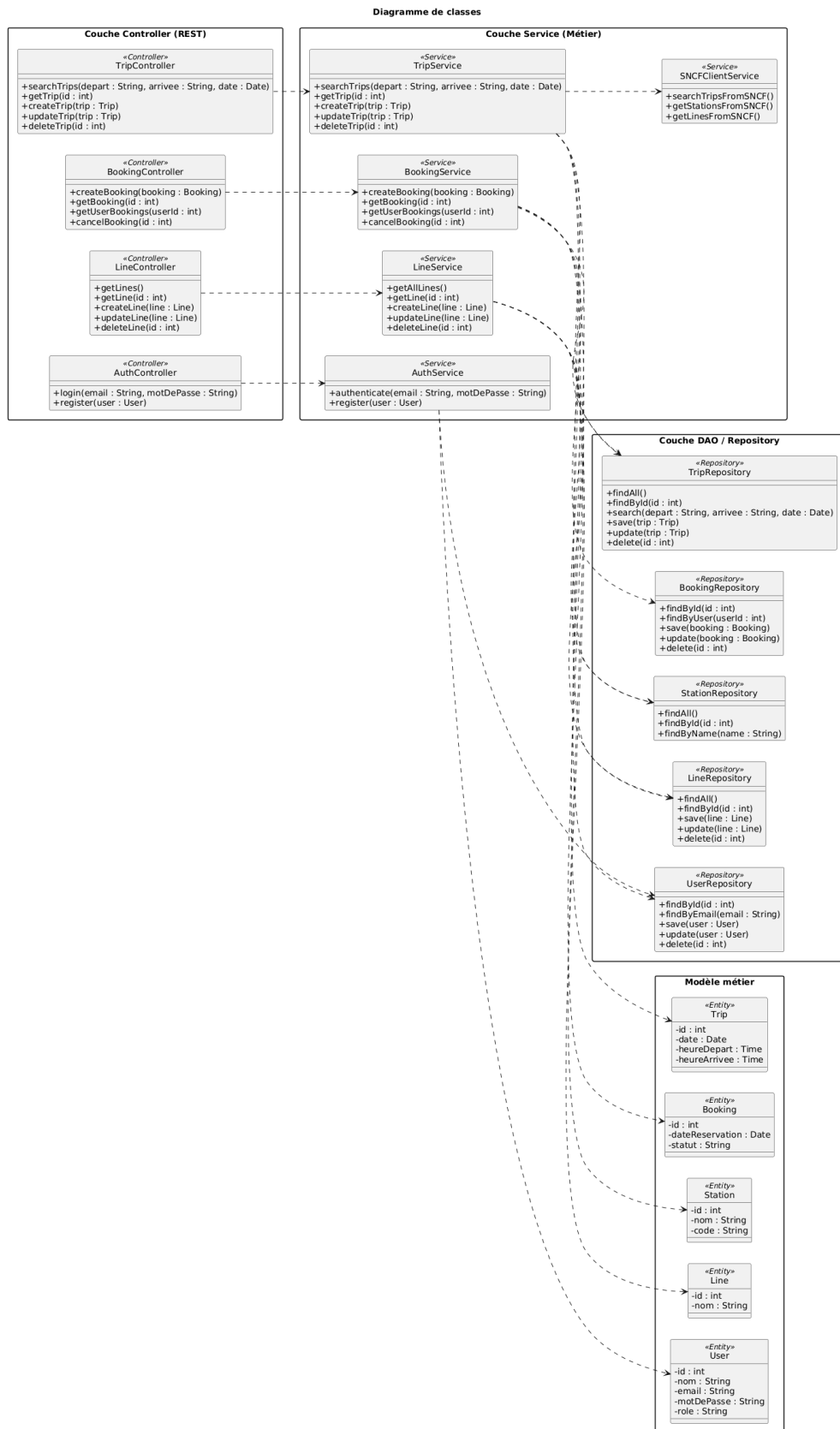


FIGURE 2.2 – Diagramme de classes

2.3 Diagramme structure projet

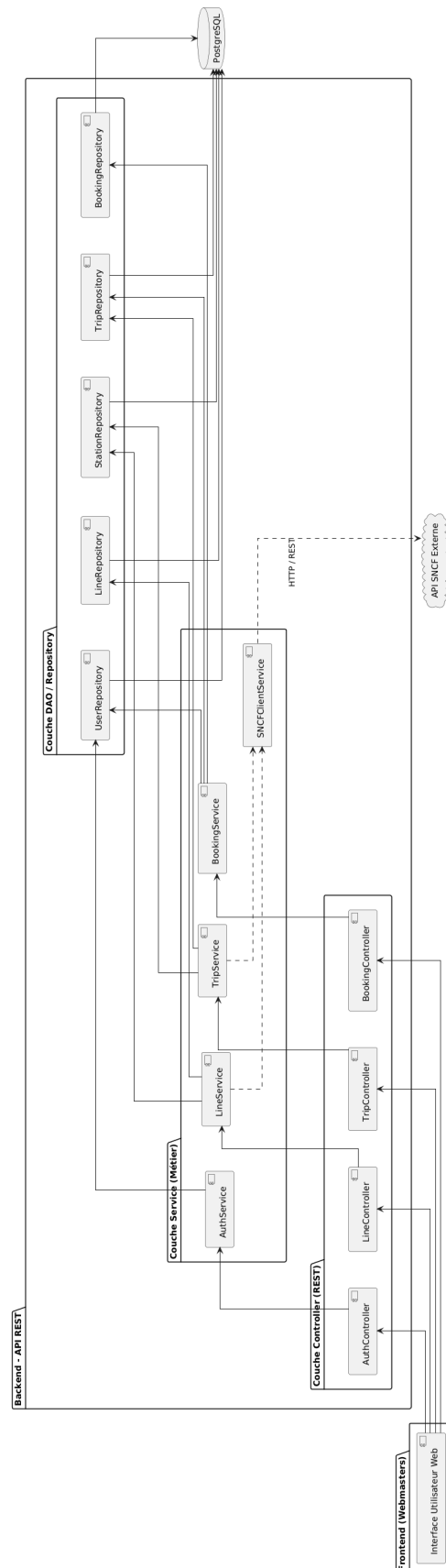


FIGURE 2.3 – Diagramme de structure du projet

Ce diagramme de structure du projet dispose de 4 couches, chacune étant relative à une tâche bien précise.

Le **Frontend (Webmasters)** envoie des requêtes au format HTTP/REST à la couche **Controller**, qui regroupe les points d'entrée de l'API : AuthController (F1), LineController (F2), TripController (F3) et BookingController (FO1); celle-ci transmet ensuite le traitement à la couche sous-jacente.

La couche **Service** fonctionne selon une logique dans laquelle chaque service correspond à un domaine fonctionnel (comptes, lignes, trajets, réservations) et s'appuie sur la couche Repository pour avoir accès aux données. **LineService** et **TripService** font également appel à **SNCFClientService**, point d'entrée unique et centralisé vers l'API SNCF externe, afin d'éviter que plusieurs services n'implémentent chacun leurs propres appels HTTP vers l'API SNCF.

La couche **DAO/Repository** est la seule à dialoguer directement avec la base de données PostgreSQL. Chaque repository (UserRepository, LineRepository, StationRepository, TripRepository, BookingRepository) est responsable d'une seule entité.

2.4 Diagramme de séquence

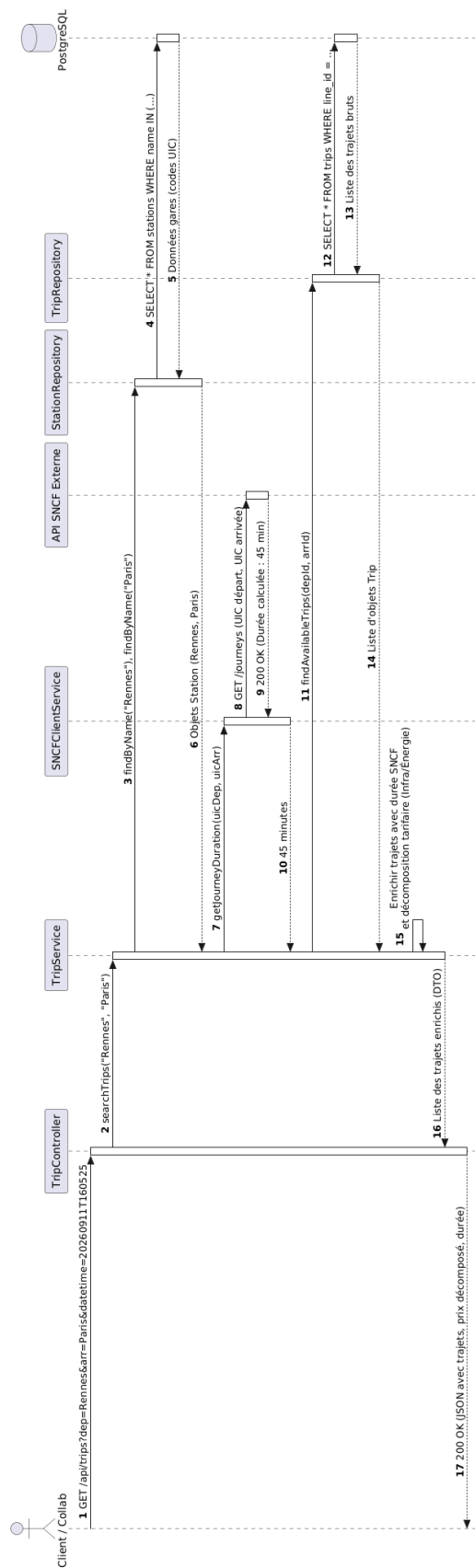


FIGURE 2.4 – Diagramme de séquence

Ce diagramme représente le fonctionnement interne du traitement d'une requête envoyée par l'utilisateur.

Nous sommes dans la situation où l'utilisateur cherche à faire un trajet entre deux gares.

La requête est d'abord envoyée vers le **Business Layer** afin de répondre à la demande par une sélection parmi les meilleurs trajets.

Le **Business Layer** commence par demander les informations de gares stockées en **backend**.

Une fois ces dernières reçues, une demande est faite à l'**API SNCF** afin d'obtenir les différents trajets et leur durée entre les gares de départ et d'arrivée.

Parallèlement, une demande sur ces mêmes trajets fournis par notre entreprise est faite en **backend**.

Il y a enfin comparaison entre les trajets fournis par l'**API SNCF** et ceux que fournit par notre entreprise afin de finaliser la liste de trajets optimaux.

La réduction liée à l'abonnement qu'a souscrit l'utilisateur est enfin appliquée au prix des différents trajets et les résultats optimaux sont envoyés à l'utilisateur

Chapitre 3

Endpoints de l'API

Ce chapitre précise la mise en œuvre des échanges décrits dans le diagramme de structure du projet : les appels vers l'API externe, centralisés dans le `SNCFClientService`, et les points d'entrée exposés par la couche `Controller`.

3.1 Endpoints consommés de l'API SNCF

Comme indiqué dans le diagramme de cas d'utilisation, la gestion des gares et des lignes ainsi que la recherche de trajets nécessitent de récupérer des informations auprès de l'API *SNCF*. Deux endpoints ont été retenus à cette fin.

L'endpoint `/places` permet de rechercher des gares par autocomplétion. Chaque résultat associe un libellé à un identifiant de la forme `stop_area:SNCF:87686006`. Il répond à l'exigence d'aide à la saisie des fonctionnalités **F2** et **F4**.

L'endpoint `/journeys` calcule les itinéraires entre deux gares et fournit la durée du trajet. Il permet à la fois de vérifier qu'une liaison ferroviaire existe et d'obtenir le temps de trajet exigé pour la fonctionnalité **F3**. Il correspond à l'appel représenté dans le diagramme de séquence, par lequel le *Business Layer* obtient la durée d'un trajet entre deux gares.

Conformément au diagramme de structure, ces appels sont regroupés dans un module unique jouant le rôle du `SNCFClientService`, afin d'éviter que plusieurs services n'implémentent chacun leurs propres requêtes vers l'API externe.

3.2 Endpoints exposés par notre API

Les endpoints développés à ce stade relèvent du `LineController` et couvrent la fonctionnalité **F2**, ainsi que l'aide à la saisie des gares commune aux fonctionnalités **F2** et **F4**.

Méthode	Chemin	Description
GET	<code>/gares</code>	Recherche de gares par autocomplétion
GET	<code>/lignes</code>	Consultation des lignes d'exploitation
POST	<code>/lignes</code>	Création d'une ligne entre deux gares
DELETE	<code>/lignes/{id}</code>	Suppression d'une ligne

TABLE 3.1 – Endpoints exposés par l'API

Ces endpoints sont accessibles au profil **COLLABORATEUR** tel que défini dans le diagramme de cas d'utilisation, à l'exception de la recherche de gares, également utilisée par le profil **CLIENT**. Le contrôle effectif des habilitations sera mis en place avec la fonctionnalité **F1**.

La documentation interactive, générée automatiquement au format *OpenAPI* et accessible à l'adresse `/docs`, permet de tester ces endpoints. Les endpoints relevant de l'`AuthController` (**F1**), du `TripController` (**F3**) et du `BookingController` (**FO1**) restent à implémenter, de même que les fonctionnalités **F4** et **F5**.